

PHP バグ・脆弱性の温床10連発

Kill your PHP code by hitting 10 combos

！ お前誰よ



- うさみけんた (@tadsan) / Zonu.EXE
- GitHub/Packagistでは id: zonuexe
- ピクシブ株式会社 pixiv事業本部
- Emacs Lisper, PHPer
 - Emacs PHP Modeのメンテナ引き継ぎました
 - 好きなリスプはEmacs Lispです
- Qiitaに記事を書いたり変なコメントしてるよ

pixiv



お絵描きがもっと楽しくなる場所を創る

pixiv 2018年度新卒
エントリー受付中



Friends of Emacs-PHP development

Join us!

<?php

- Repositories 25
- People 5
- Teams 1
- Projects 0
- Settings

Find a repository... Type: All Language: All

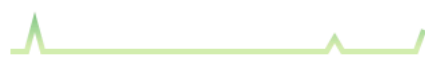
Customize pinned repositories New

php-runtime.el

PHP-Emacs bridge, call PHP function from Emacs

- php
- emacs
- melpa

Emacs Lisp 2 1 GPL-3.0 Updated 2 days ago



phpactor.el

Interface to Phpactor (an intelligent code-completion and refactoring tool for PHP)

Emacs Lisp 8 4 1 issue needs help Updated 3 days ago



php-mode

A PHP mode for GNU Emacs



Top languages

- Emacs Lisp
- PHP

Most used topics

emacs melpa php major-mode

Manage

People 5 >



Invite someone



Search or jump to...



[Pull requests](#)

[Issues](#)

[Marketplace](#)

[Explore](#)



emacs-jp

📍 Japan

🔗 <http://emacs-jp.github.com/>

Repositories 18

People 37

Teams 8

Projects 0

Settings

Find a repository...

Type: All ▾

Language: All ▾

[Customize pinned repositories](#)

New

reading-init.el

init.el読書会のページ

[html](#)

[dotfiles](#)

[emacs](#)

[emacs-lisp](#)

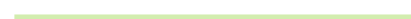
● Ruby ★ 6 🍴 1 Updated 6 days ago



helm-c-yasnippet

Helm source for yasnippet

● Emacs Lisp ★ 14 🍴 5 Updated on 21 Mar



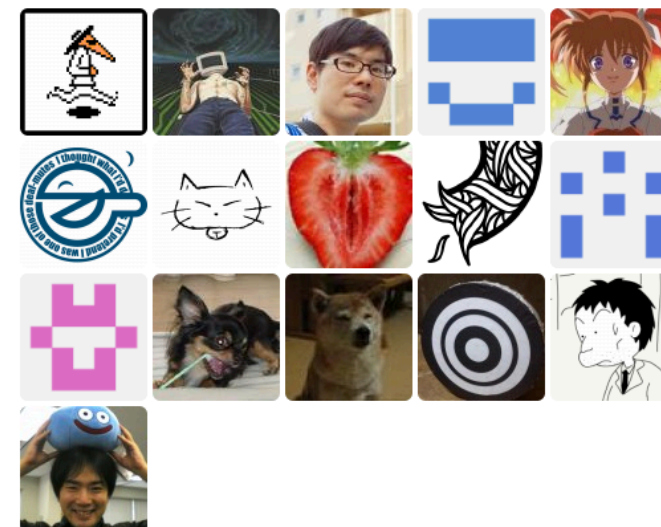
replace-colorthemes

Top languages

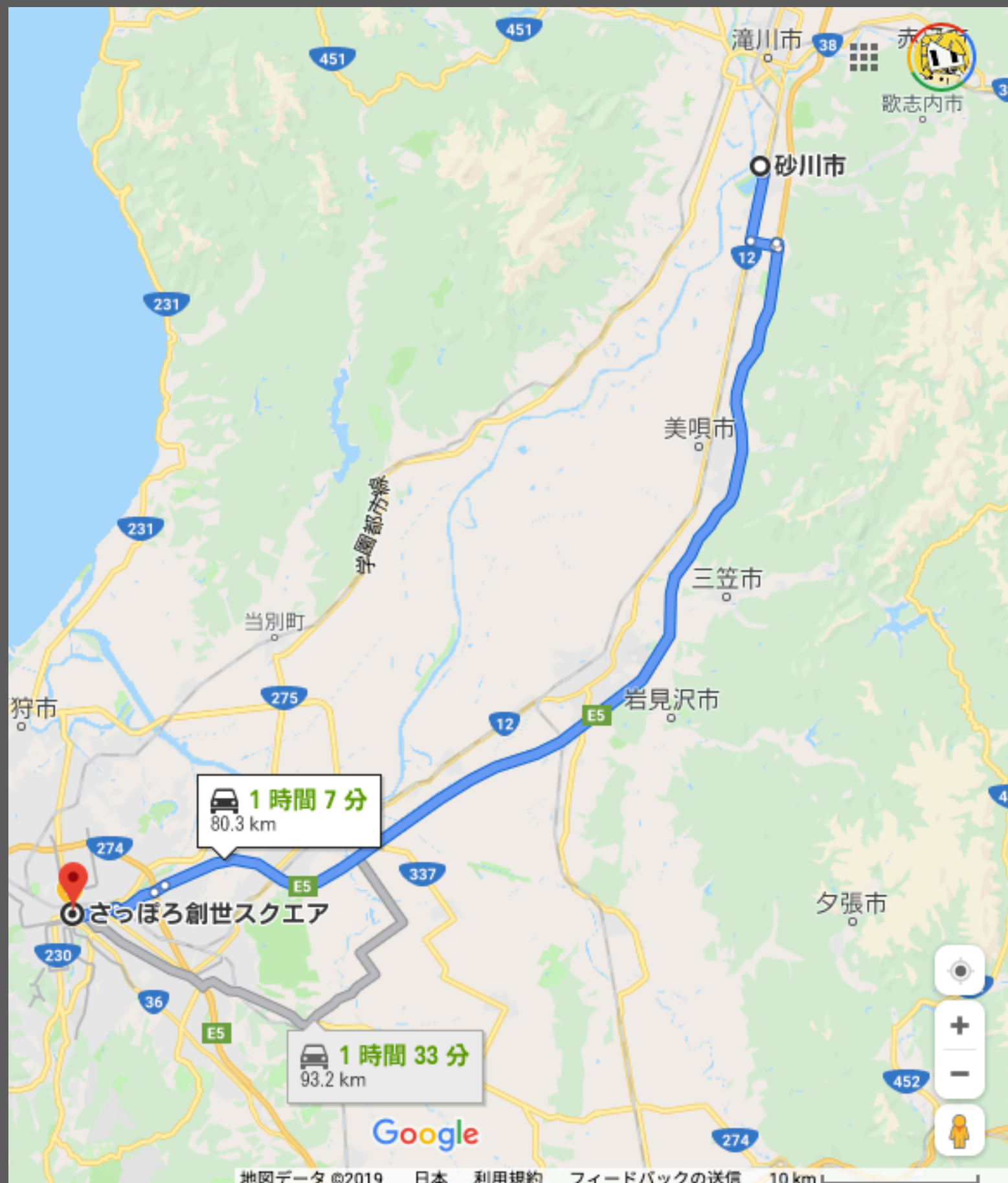
● Emacs Lisp ● Ruby ● TeX

People

37 >



実家は近所
です！！！！



前回のぺちこん
北海道2016が
カンファレンス
デビュー戦

ついでにその前
は札幌市中央区
Ruby会議01

さて

PHPカンファレンス
は楽しかったですか！

ここで学んだことは
今後の仕事に活か
していきましょう

本題

セッションの概要 *

PHPはそれ自体がWebに特化した機能を持つ開発環境ですが、使用方法を誤ると予期せず実行が中断されたり、脆弱性の温床になるようなパターンがいくつもあります。このトークでは時間の許す限りパターンの紹介と解決策の提案を紹介します。(タイトルの10の進数表記は任意の整数とする)

セッションの長さ *

- ☐ 45分
- ☒ 30分
- ☐ LT (5分)

ということで
クイックに
いきましょう

10連発

ただしタイトル
「10連発」は任意
の進数表記とする

PHP クイック バグ・脆弱性の温床10連発

Kill your PHP code by hitting 10 combos

| 例題1

```
<?
$id = $_REQUEST['id'];
if (!is_numeric($id))
    printf("<p>{$id}は正しいIDではありません</p>");
?>
```

例題1

short open

tagはphp.iniの設定依存なので、
基本は <?php または <?=
<?>

```
<?
$id = $_REQUEST['id'];
if (!is_numeric($id))
    printf("<p>{$id}は正しいIDではありません</p>");
?>
```

例題1

short open

tagはphp.iniの設定依存かつ、

基本的に

php.ini依存かつ、

\$_GETと\$_POSTが混ざると不適切な取り扱いされる傾向がある

```
<?
$id = $_REQUEST['id'];
if (!is_numeric($id))
    printf("<p>{$id}は正しいIDではありません</p>");
?>
```

例題1

short open

tagはphp.iniの設定依存かつ、

基本的に

php.ini依存かつ、

\$_GETと\$_POSTが混ざると不適切な取り扱いされる傾向がある

```
<?
```

```
$id = $_REQUEST['id'];
```

```
if (!is_numeric($id))
```

```
    printf("<p>{$id}は正しいIDではありません</p>");
```

```
?>
```

何を期待するのか

不明瞭だがMySQLのAUTO_INCREMENT
なサロゲートキーなら不適切

例題1

short open

tagはphp.iniの設定依存かつ、

基本的に

php.ini依存かつ、

\$_GETと\$_POSTが混ざると不適切な取り扱いされる傾向がある

```
<?
```

```
$id = $_REQUEST['id'];
```

```
if (!is_numeric($id))
```

```
    printf("<p>{$id}は正しいIDではありません</p>");
```

```
?> ,
```

何を期待するのか

不明瞭だがMySQLのAUTO_INCREMENT
なサロゲートキーなら不適切

printfフォーマットに
変数埋め込みをしない (入力値に%が含まれる
とWarningが発生)

例題1

short open

tagはphp.iniの設定依存かつ、

基本的に

php.ini依存かつ、

\$_GETと\$_POSTが混ざると不適切な取り扱いされる傾向がある

<?

```
$id = $_REQUEST['id'];
```

```
if (!is_numeric($id))
```

```
    printf("<p>{$id}は正しいIDではありません</p>");
```

?> ,

何を期待するのか

不明瞭だがMySQLのAUTO_INCREMENT
なサロゲートキーなら不適切

printfフォーマットに

数埋め込みをしない (入力値に%が含まれる
とWarningが発生)

脆弱性ではないが変な文字が混
ざってるので ?> は消す

例題1

short open

tagはphp.iniの設定依存かつ、

基本的に

php.ini依存かつ、

\$_GETと\$_POSTが混ざると不適切な取り扱いされる傾向がある

<?

```
$id = $_REQUEST['id'];
```

```
if (!is_numeric($id))
```

```
    printf("<p>{$id}は正しいID
```

?> ,

脆弱性じゃないけど { は一文でも几帳面に付ける方がオススメ

何を期待してるの?

不明瞭だがMySQLのAUTO_INCREMENT
なサロゲートキーなら不適切

printfフォーマットに

脆弱性ではないが変な文字が混ざってるので ?> は消す

数埋め込みをしない (入力値に%が含まれるとWarningが発生)

改善案は各自へ
の宿題とします

| 例題2

```
const VALIDATE_PATTERN = "/
    ^                                # begin
    ([a-zA-Z0-9])+([a-zA-Z0-9\._-])*  # localpart
    @                                # at mark
    ([a-zA-Z0-9_-])+([a-zA-Z0-9\._-]+)+ # host
    $                                # end
/x";

$email = filter_input(INPUT_GET, 'email');
if (!preg_match(VALIDATE_PATTERN, $email)) {
    echo "Bad mail address!!";
}
register_account(strtolower($email), $password);
```

例題2

PCRE正規表現は /x 修飾子を付ける
と空白が無視されるので、これはGood

```
const VALIDATE_PATTERN = "/
    ^                                # begin
    ([a-zA-Z0-9])+([a-zA-Z0-9\._-])*  # localpart
    @                                # at mark
    ([a-zA-Z0-9_-])+([a-zA-Z0-9\._-]+)+ # host
    $                                # end
/x";

$email = filter_input(INPUT_GET, 'email');
if (!preg_match(VALIDATE_PATTERN, $email)) {
    echo "Bad mail address!!";
}
register_account(strtolower($email), $password);
```


例題2

PCRE正規表現は /x 修飾子を付ける
と空白が無視されるので、これはGood

だが、バリデーション(妥当性検証)の
ために正規表現で判定するのはだめ!!!

```
const VALIDATE_PATTERN =
    ^
    ([a-zA-Z0-9])+([a-zA-Z0-9\._-]) # localpart
    @
    ([a-zA-Z0-9_-])+([a-zA-Z0-9\._-])+ # host
    $
    # end

/x";

$email = filter_input(INPUT_GET, 'email');
if (!preg_match(VALIDATE_PATTERN, $email)) {
    echo "Bad mail address!!";
}
register_account(strtolower($email), $password);
```

例題2

PCRE正規表現は /x 修飾子を付ける
と空白が無視されるので、これはGood

だが、バリデーション(妥当性検証)の
ために正規表現で判定するのはだめ!!!

FILTER_VALIDATE_EMAILという
フラグを使えば取得と同時に検証できる

```
const VALIDATE_PATTERN =  
    ^  
    ([a-zA-Z0-9]+)([a-zA-Z0-9\._-]+)@  
    ([a-zA-Z0-9_-]+)([a-zA-Z0-9_-]+).  
    $  
/x";  
  
$email = filter_input(INPUT_GET, 'email');  
if (!preg_match(VALIDATE_PATTERN, $email)) {  
    echo "Bad mail address!!";  
}  
register_account(strtolower($email), $password);
```


例題2

PCRE正規表現は /x 修飾子を付ける
と空白が無視されるので、これはGood

だが、バリデーション(妥当性検証)の
ために正規表現で判定するのはだめ!!!

```
const VALIDATE_PATTERN =  
    ^  
    ([a-zA-Z0-9])+([a-zA-Z0-9\._-])*\slocalpart  
    @  
    ([a-zA-Z0-9])+([a-zA-Z0-9\._-])*\sdomainpart
```

Bad って表示してるだけで、処理は
停止できてない!!!!

FILTER_VALIDATE_EMAILという
関数を使えば取得と同時に検証できる

```
$email = trim($_POST[INPUT_GET, 'email']);  
if (!preg_match(VALIDATE_PATTERN, $email)) {  
    echo "Bad mail address!!";  
}  
register_account(strtolower($email), $password);
```

例題2

PCRE正規表現は /x 修飾子を付ける
と空白が無視されるので、これはGood

だが、バリデーション(妥当性検証)の
ために正規表現で判定するのはだめ!!!

```
const VALIDATE_PATTERN =  
    ^  
    ([a-zA-Z0-9]+)([a-zA-Z0-9\._-]+)@  
    ([a-zA-Z0-9-]+\.[a-zA-Z0-9-]+)
```

Bad って表示してるだけで、処理は
停止できてない!!!!

FILTER_VALIDATE_EMAILという
関数を使えば取得と同時に検証できる

```
$email = trim($_POST[INPUT_GET, 'email']);  
if (!preg_match(VALIDATE_PATTERN, $email)) {  
    echo "Bad mail address!!";  
}  
register_account($email, $password);
```

正規化

は検討が必要(本来ローカルパートは大文
字小文字区別するが、実際はサービスの実
装依存)

| 例題2++

// これならどうだ！

```
$email = filter_input(INPUT_GET, 'email',  
                      FILTER_VALIDATE_EMAIL);  
if ($email !== false) {  
    echo "Bad mail address!!";  
    exit;  
}  
  
[$local, $host] = explode('@', $email);  
$normalized = $local . '@' . strtolower($host);  
  
register_account($email, $password);
```


| 例題2++

// これならどうだ！

```
$email = filter_input(INPUT_GET, 'email',  
                      FILTER_VALIDATE_EMAIL);  
if ($email !== false) {  
    echo "OK";  
    exit;  
}  
  
[ $local, $normalized ] = dns_get_record($host);  
  
register_account($email, $password);
```

それでも

良くないことがあるので続きは懇親

会で！！！！！！！！